



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ
НА ТЕМУ:
«Расширение суперкомпилятора
модельного языка»

Студент ИУ9-82Б
(Группа)

(Подпись, дата)

Н. О. Горбатов
(И.О. Фамилия)

Руководитель ВКР

(Подпись, дата)

А. В. Коновалов
(И.О. Фамилия)

Консультант

(Подпись, дата)

(И.О. Фамилия)

Консультант

(Подпись, дата)

(И.О. Фамилия)

Нормоконтролер

(Подпись, дата)

(И.О. Фамилия)

2024 г.

АННОТАЦИЯ

Суперкомпиляция, как мощный метод трансформации программ, актуальна для повышения производительности и снижения ресурсозатрат современных сложных программ, обеспечивая их корректность и эффективность. В данной работе исследуются существующие подходы к специализации программ, анализируются выходные форматы функций, и разрабатывается алгоритм построения выходных форматов онлайн. Созданный суперкомпилятор, включающий генератор остаточной программы и использующий информацию о выходных форматах конфигураций, был успешно протестирован и показал свою эффективность в задачах специализации и анализа свойств программ.

Расчётно-пояснительная записка выпускной квалификационной работы на тему: «Расширение возможностей суперкомпиляции модельного языка» состоит из 53 страниц, содержит 9 разделов, 27 листинга, 3 таблицы и 7 рисунков. В процессе выполнения было использовано 15 источников.

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ	8
ВВЕДЕНИЕ	10
1 Модельный язык	12
2 Принцип работы суперкомпилятора	14
3 Выходные форматы конфигураций и декомпозиция функций	16
4 Обзор результатов в области специализации программ	18
4.1 Суперкомпилятор SCP4	18
4.2 Другие разновидности специализации	19
5 Описание исходного суперкомпилятора	21
5.1 Отношение гомеоморфного вложения	21
5.2 Локальное и глобальное заикливание	22
5.3 Наиболее тесное обобщение	23
5.4 Прогонка	24
5.5 Основные классы и модули	25
6 Проектирование усовершенствованного суперкомпилятора .	27
6.1 Создание let-узлов	27
6.2 Обработка let-узлов	28
6.3 Проверка форматных гипотез	30
6.4 Генерация остаточной программы	31
7 Реализация	32
7.1 Графическое отображение дерева конфигураций	32
7.2 Преобразования основного цикла в рекурсивную функцию	32
7.3 Реализация функции поиска обобщённого вложения	36
7.4 Реализация генератора остаточной программы	37
8 Тестирование	41

9	Руководство пользователя	46
	ЗАКЛЮЧЕНИЕ	47
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	48
	ПРИЛОЖЕНИЕ А	50

ОПРЕДЕЛЕНИЯ

Определение 1. Конструктор — именованный кортеж выражений. Все обрабатываемые программой¹ данные являются деревьями, построенными с помощью конструкторов.

Определение 2. Параметр — заведомо неизвестные данные. За параметром может скрываться произвольное дерево конструкторов.

Определение 3. Выражением общего вида (далее просто выражением) называется любая возможная композиция функций, конструкторов и параметров. Если в выражении присутствуют вызовы функций, такое выражение называется активным, если отсутствуют — пассивным.

Определение 4. Конфигурация — изображение множества состояний вычислительного процесса, задающееся выражением общего вида.

Определение 5. Граф конфигураций (вычислительный граф) — дерево с обратными стрелками, узлы которого являются конфигурациями, а дуги соответствуют различным операциям: шагу вычисления, сужению, подстановке, замене конфигурации или зацикливанию.

Определение 6. Выходной формат конфигурации — пассивное выражение, аппроксимирующее множество результатов вычислительных процессов, соответствующих конфигурации.

Определение 7. Подстановка — набор подстановочных правил для выражения, ставящих в соответствие параметрам некоторые новые подвыражения.

Определение 8. let-узел — конфигурация, состоящая из тела и подстановки. Обычно используется для обобщения конфигураций и декомпозиции конструкторов.

Определение 9. Прогонка — обобщённый шаг вычисления конфигурации, который заключается в рассмотрении всех возможных значений параметров. В общем случае приводит к ветвлению графа конфигураций.

¹Написанной на модельном языке, входном языке суперкомпилятора

Определение 10. Обобщение — умышленный отказ от дополнительной информации о состояниях вычислительно процесса с целью свести ситуацию к ранее уже рассмотренной конфигурации.

Определение 11. Остаточная программа — программа, являющаяся результатом работы суперкомпилятора.

ВВЕДЕНИЕ

В период 1960-70-х годов произошло зарождение большинства методов анализа и трансформации программ, актуальных и поныне. Один из выдающихся разработчиков того времени, Валентин Федорович Турчин (1931-2010), предложил уникальную концепцию, получившую название «суперкомпиляция». Эта концепция — результат применения более общей идеи «метасистемных переходов», изложенной Турчиным в его работе «Феномен науки».

Суперкомпиляция — особый вид метавычислений, применяющийся для решения множества задач, среди которых обычно выделяют:

- оптимизацию и специализацию программ (повышение скорости работы и уменьшение размера программ);
- анализ свойств программ и верификацию;
- инверсию программ.

Суперкомпиляция является актуальной темой выпускной квалификационной работы благодаря её большому потенциалу в решении перечисленных задач. Современные программы становятся всё более сложными, и задачи по обеспечению их корректности и эффективности выходят на первый план. Суперкомпиляция, как мощный метод трансформации программ, позволяет выявлять и устранять избыточные вычисления, повышая производительность и снижая ресурсозатраты. Кроме того, она способствует автоматическому анализу и доказательству корректности программного кода, что критически важно для верификации программ, особенно в областях, требующих высокой надёжности, таких как авиация, медицина и банковский сектор. В условиях быстро меняющихся требований и среды выполнения программ, суперкомпиляция также поддерживает гибкость за счёт специализации программ под конкретные задачи и условия, повышая их эффективность.

В рамках этой работы с помощью метода суперкомпиляции будет, во-первых, проводиться оптимизация программ путём специализации и нахождения композиций функций, и во-вторых, для каждого состояния вычислительно процесса будет построена аппроксимация результата вычисления, что позволит осуществлять дополнительную оптимизацию, а также делать выводы о свойствах программ.

Ранее мной уже был разработан суперкомпилятор модельного функционального языка первого порядка. В настоящей работе предлагается расширить возможности суперкомпиляции путём анализа выходных форматов функций (конфигураций).

В своей работе «О суперкомпиляции» [1] Андрей П. Немытых отмечал, что о методах оптимизации можно говорить только в контексте конкретной модели вычислений или, в более технических терминах, в контексте конкретной реализации языка программирования. Поэтому разумным кажется начать с описания входного модельного языка суперкомпилятора.

1 Модельный язык

В качестве входного модельного языка было выбрано подмножество Scala с case-классами и частичными функциями. Язык включает в себя определение case-классов, case-объектов и методов единственного объекта Main.

Case-классы неявно наследуют Any, содержат поля типа Any и не имеют тела. Пример объявления case-класса и case-объекта приведён в листинге 1.

Листинг 1: Пример объявления case-класса и case-объекта

```
case object C
case class Cons(car: Any, cdr: Any)
```

Методами класса Main могут быть только частичные функции типа $(Any, \dots, Any) \Rightarrow Any$. В образцах могут использоваться только case-классы и case-объекты, определённые в текущей программе, а также имена переменных. В выражениях могут использоваться те же конструкции, что и в образцах плюс вызовы функций.

Синтаксис модельного языка приведён в листинге 2.

Листинг 2: Синтаксис модельного языка в РБНФ

```
<program>      ::= { <case_object> | <case_class> | <object_main> }.
<case_object>  ::= "case" "object" <ident>.
<case_class>   ::= "case" "class" <ident> "(" [ <ident> ":" "Any" { "," <ident> ":" "Any" } ]
  ↪ ")".
<object_main>  ::= "object" "Main" "{" { <func_def> } "}".
<func_def>     ::= "def" <ident> ":" <case_sign> "=>" "Any" "=" "{" { <rule> } "}".
<rule>         ::= "case" <pattern_list> "=>" <expr>.
<pattern_list> ::= "(" <expr> { "," <expr> } ")" | <expr>.
<expr>         ::= <call> | <varOrCtr>.
<call>         ::= ident "(" [ <expr> { "," <expr> } ] ")".
<case_sign>    ::= "(" "Any" { "," "Any" } ")" | "Any".
```

Таким образом, перед нами простой функциональный язык первого порядка, в котором, например, нет предопределённого понятия числа. Все данные, с которыми работает программа, представляют собой так называемые конструкторы. Конструкторы могут иметь произвольную вложенность и быть сколь угодно сложно вложенными друг в друга, образуя тем самым древовидную структуру.

В листинге 3 приведена реализация на модельном языке классического примера на суперкомпиляцию. Функция `main(x)` должна заменять в списке и A, и B на C.

Листинг 3: Пример программы модельного языка

```
case object A
case object B
case object C
case object Nil_
case class Cons(head: Any, tail: Any)

object Main {
  def fab: Any => Any = {
    case Cons(A, xs) => Cons(B, fab(xs))
    case Cons(x, xs) => Cons(x, fab(xs))
    case Nil_ => Nil_
  }

  def fbc: Any => Any = {
    case Cons(B, xs) => Cons(C, fbc(xs))
    case Cons(x, xs) => Cons(x, fbc(xs))
    case Nil_ => Nil_
  }

  def main: Any => Any = {
    case xs => fbc(fab(xs))
  }
}
```

У исходного суперкомпилятора входной язык совпадает с выходным, однако, в актуальной работе необходимость использования выходных форматов конфигураций при генерации остаточной программы вынудила расширить выходной язык (см. Раздел 6.4).

2 Принцип работы суперкомпилятора

В статье «Суперкомпиляция: основные принципы и базовые понятия» [2] вводится следующее определение для суперкомпилятора: пусть P — исходная программа, а Γ — ограничения на условия эксплуатации P , тогда суперкомпилятор (или специализатор) — программа, принимающая на вход пару (P, Γ) и порождающая на выходе остаточную программу P' , удовлетворяющую следующим требованиям:

- P' эквивалентна P , если выполнены условия Γ .
- P' получается путём устранения из P тех частей и действий, которые становятся ненужными в результате наложения условий Γ .

Надежды и ожидания, связанные со специализацией P , состоят в следующем:

- P' будет эффективнее, чем P .
- P' будет меньше, чем P .
- P' будет проще, чем P .

Под программой P будем понимать набор правил редукции вида $f(\alpha_1, \alpha_2, \dots, \alpha_n) \rightarrow \beta$, где f — некоторая функция; α_i — образцы для сопоставления (выражения, состоящие только из конструкторов и переменных); β — результат редукции (выражение, допускающее, помимо конструкторов и переменных, ещё и вызовы функций). Образцы не могут содержать повторяющиеся переменные (ограничение, сильно упрощающее суперкомпиляцию). Выражение в правой части не может содержать «необъявленные» переменные. Поскольку области сопоставления правил могут пересекаться, чтобы не возникало конфликтов и недетерминированности, у правил должен быть указан приоритет.

Одних правил редукции недостаточно для выполнения программы — не хватает точки входа. Такой точкой входа будем считать некоторое выражение Ξ , содержащее вызовы функций, конструкторы, но не параметры. Если параметризовать Ξ , то из точки входа получится Γ — ограничение на условия эксплуатации.

Выполнение программы P (считаем, что язык «ленивый») — детерминированный процесс последовательного применения правил редукции к Ξ , результатом которого может быть либо ошибка, либо выражение ξ , не содержащее вызовов функций (ξ будет содержать только конструкторы).

В наших терминах суперкомпиляция — попытка редуцировать выражение Γ в общем виде, или анализ поведения программы на всех входных данных при условии Γ . Если можно сразу применить правило, суперкомпилятор делает это, однако регулярно возникают ситуации, когда редукции препятствуют неизвестные параметры или вызовы функций. В первом случае необходимо найти «сужения» (подстановки) для параметров, которые позволят использовать правила. Этот процесс называется «прогонкой». Во втором случае редуцировать придётся сначала вложенный вызов.

Результат шага прогонки конфигурации α в общем случае — набор конфигураций, на которые будет ветвиться α . Здесь и возникает необходимость строить «граф конфигураций» (вычислительный граф). Стрелки, связывающие узлы дерева, соответствуют каким-то действиям и сужениям.

Если исходная программа содержит циклы и/или рекурсию, то дерево конфигураций получается бесконечным. Поэтому в процессе суперкомпиляции делается попытка свернуть бесконечное дерево в конечный граф конфигураций. Для этого конфигурации сравниваются между собой. Если появляются две «похожие» конфигурации, делается попытка «свести» ту конфигурацию, что находится в дереве ниже, к той, что появилась выше, эта операция называется «обобщение». В результате в дереве появляется «обратная» стрелка, и дерево превращается в граф с циклами.

Если в процессе суперкомпиляции удаётся построить граф конфигураций, который содержит полную информацию о возможных путях вычисления, тогда эту информацию можно превратить в программу (конечного размера). Эта программа, называемая «остаточной» и будет окончательным результатом работы суперкомпилятора.

Каждый базисный узел ¹ в графе помечается неким идентификатором, который будет служить именем функции. Параметры, встречающиеся в конфигурации, становятся аргументами функции. Сужения, полученные в ходе прогонки, преобразовываются в правила редукции новой программы.

¹Базисный узел — вершина, в которую ведёт хотя бы одна обратная стрелка.

3 Выходные форматы конфигураций и декомпозиция функций

Определим $\text{RANGE}(E)$ как множество всех возможных рабочих конфигураций (т.е. конфигураций без вызовов функций и переменных), которые могут получаться из конфигурации E .

Пусть существует $\text{Out}F$, такая что для любого $r \in \text{RANGE}(E)$ существует подстановка σ , такая что $r = \text{Out}F/\sigma$. Таким образом, $\text{Out}F$ будем называть выходным форматом конфигурации E .

Известно, что суперкомпиляция эффективно справляется с вычислением композиций функций. Например, композиция функций $F(\dots)$ и $G(\dots)$, может быть заменена на одну функцию $FG(\dots)$ в оптимизированной программе. При этом данные, требуемые для промежуточных структур, порождаемых вызовом $G(\dots)$ и необходимые для вызова $F(\dots)$, в оптимизированной программе могут быть опущены. Разновидность суперкомпиляции, фокусирующаяся на оптимизации подобных композиций, носит название дефорестации [3].

В работе Берстолла и Дарлингтона [4], представлены принципы для эквивалентного преобразования кода программ. Правила, описанные в этой публикации, характеризуются своей обратимостью. Это означает, что из программы P_2 можно получить P_1 , используя эти правила, и наоборот, преобразовать P_1 в P_2 путём эквивалентного преобразования.

Необходимо использовать определённую стратегию для того, чтобы такие преобразования выполнялись без участия человека. Суперкомпиляция представляет собой один из методов, позволяющих достичь этого.

Представим, что первичная программа P_1 включала в себя последовательность вызовов функций в виде $F(G(\dots))$, а в результате эквивалентных трансформаций мы получили программу P_2 , содержащую вызов $FG(\dots)$. Тогда, выполнив инверсию преобразований, можно декомпонировать $FG(\dots)$ обратно в исходную последовательность вызовов $F(G(\dots))$. Такой процесс, при котором вызов $F(\dots)$ преобразуется в исходную серию вызовов вида $F'(F''(\dots))$, получил наименование «декомпозиция».

Найденные выходные форматы конфигураций позволяют осуществить декомпозицию функций на этапе построения остаточной программы следующим

образом: внутренняя функция вычисляет переменные формата, а внешняя – подставляет посчитанные значения в формат.

4 Обзор результатов в области специализации программ

4.1 Суперкомпилятор SCP4

Важным достижением в области суперкомпиляции стало появление экспериментального суперкомпилятора SCP4 [5]. Все предшествующие суперкомпиляторы были предназначены для простейших чисто теоретических языков, в то время как SPC4 — для реального языка программирования РЕФАЛ-5 [6]. Кроме того в SPC4 был реализован простейший алгоритм построения индуктивных выходных форматов¹ конфигураций: выходной формат строится только инструментами обобщения (см. раздел 5.3), которые применяются последовательно к концам ветвей, — обобщая выходной формат ранее рассмотренных ветвей и описания конца текущей ветви [5]. Однако происходит это уже после построения компоненты факторизации², что ограничивает глубину анализа.

Автор и основной разработчик SCP4 А. П. Немытых в своей работе «Суперкомпилятор SCP4: Общая структура» [5] рассматривает следующий пример (листинг 4):

Листинг 4: Пример на суперкомпиляцию: программа на языке РЕФАЛ-5

```
$ENTRY Go { e.number = <UnarySum (I I I) (e.number)>; }
UnarySum {
  (e.numb1) (I e.numb2) = I <UnarySum (e.numb1) (e.numb2)>;
  (e.numb1) () = e.numb1;
}
```

Определение функции UnarySum, рассмотренное отдельно от контекста вызова функции, не несёт никакой информации о структуре выхода этой функции. После специализации по контексту вызова **без учёта** выходного формата могла бы получиться нижеследующая программа³ (листинг 5):

¹Т.е. выходных форматов, найденных посредством выдвижения гипотезы и её доказательства по индукции.

²Компонента факторизации — это подграф графа конфигураций, лежащий ниже базисной вершины.

³Вместо остаточной программы тут следовало бы предъявить компоненту факторизации в терминах языка РЕФАЛ-графов, поскольку рассматривается промежуточный этап — сразу после построения факторизации и перед нахождением выходного формата. Однако, чтобы не усложнять изложение, в листинге приведена остаточная программа, соответствующая интересующей нас компоненте.

Листинг 5: Результат специализации без учёта выходного формата

```
$ENTRY Go { e.41 = <F7 e.41>; }  
F7 {  
    I e.41 = I <UnarySum e.41>;  
    /* пусто */ = I I I;  
}
```

Алгоритм построения выходных форматов обобщит выходы «I <UnarySum e.41>» и «I I I» функции F7 до выходного формата «I e.102», что позволит упростить программу (листинг 6) после вынесения «наружу» из функции F7 неизменного префикса.

Листинг 6: Результат специализации с учётом выходного формата

```
$ENTRY Go { e.41 = I <F7 e.41>; }  
F7 {  
    I e.41 = I <UnarySum e.41>;  
    /* пусто */ = I I;  
}
```

Как замечает автор, алгоритм строит индуктивный выходной формат «I e.102», хотя более точный анализ (семантический) мог бы дать идеальный вариант «I I I e.102» и соответствующую остаточную программу (листинг 7).

Листинг 7: Желаемый результат специализации

```
$ENTRY Go { e.41 = I I I <F7 e.41>; }  
F7 {  
    I e.41 = I <UnarySum e.41>;  
    /* пусто */ = /* пусто */;  
}
```

4.2 Другие разновидности специализации

Помимо суперкомпиляции существуют и другие подходы к решению задачи специализации. Например, предложенная Н. Д. Джонсом (Дания), а впоследствии хорошо разработанная, техника «частичных вычислений» [7]. Изначальная идея заключалась в разделении специализации на две стадии: связывание по времени (ВТА¹) и собственно специализацию. Это позволяло эффективно решать проблемы, связанные с определением, какие части программы могут быть вычислены

¹Binding time analysis.

статически, а какие требуют динамической обработки, уменьшая тем самым сложность и размер конечного кода.

Ещё один метод специализации «обобщённые частичные вычисления» (GPC¹) был описан Ё. Футамурой в 1980-х [8]. В отличие от частичных вычислений и суперкомпиляции, подход обобщённых частичных вычислений к специализации незамкнут. Например, представленный в 2002 году [9] экспериментальный полуавтоматический специализатор WSDFU обращается к внешней системе доказательств теорем TPU [10] и внешней базе знаний для доказательств свойств преобразуемых программ. Особенный интерес представляют примеры специализации программ с числовыми аргументами, в которых в результате обобщённых частичных вычислений понижается порядок временной сложности [11].

¹Generalized partial computation.

5 Описание исходного суперкомпилятора

Рассмотрим разработанный ранее в рамках курсовой работы суперкомпилятор. Этот реализованный на Python3 суперкомпилятор принимает на вход программу модельного языка (подмножество языка Scala), анализирует её, строит дерево конфигураций и генерирует на выходе эквивалентную, но, в некотором смысле, упрощённую, более эффективную программу. В этом разделе будет приведено описание основных функциональных элементов исходного суперкомпилятора без углубления в детали реализации, а также будут рассмотрены общие алгоритмы, используемые в суперкомпиляции.

Конфигурации разбиваются на три непересекающихся подмножества: «тривиальные», «глобальные» и «локальные». Решение об обобщении принимается суперкомпилятором на основе поиска гомеоморфного вложения конфигураций одной категории. Кроме того, в суперкомпиляторе реализована разметка по имени прогоняемой функции. Таким образом, обобщению подлежат гомеоморфно вложенные конфигурации, принадлежащие одной категории, с одной и той же прогоняемой функцией.

5.1 Отношение гомеоморфного вложения

Отношение гомеоморфного вложения (обозначается $\alpha \sqsubseteq \beta$, а читается « α гомеоморфно вложено в β ») задают тремя индуктивными правилами:

1. $x \sqsubseteq y$ для любых параметров x и y .
2. $X \sqsubseteq f(Y_1, \dots, Y_k)$, если f — имя конструктора или функции и для некоторого i выполнено $X \sqsubseteq Y_i$.
3. $f(X_1, \dots, X_k) \sqsubseteq f(Y_1, \dots, Y_k)$, если f — имя конструктора или функции и для всех i выполнено $X_i \sqsubseteq Y_i$.

Или менее формально: $\alpha \sqsubseteq \beta$, когда из β можно получить α «стерев» некоторые его части¹.

Крускал также доказал [13], что для любой бесконечной последовательности конфигураций $X_1, X_2, X_3, \dots$, обязательно найдутся такие i и j , что $i < j$ и $X_i \sqsubseteq X_j$. Это означает, что для каждой полученной в процессе прогонки бесконечной последовательности конфигураций рано или поздно возникает ситуация,

¹Первыми такое отношение предложили Хигман [12] и Крускал [13].

когда некоторая конфигурация окажется вложена в одну из последующих конфигураций. Этот факт в конечном итоге гарантирует конечность графа конфигураций.

5.2 Локальное и глобальное заикливание

Применение обобщений приводит к уменьшению количества анализируемых конфигураций за счет объединения различных множеств состояний в одну, более общую, иногда с включением дополнительных, несущественных состояний. Суперкомпиляция становится менее разборчивой, что делает результат менее интересным.

Избегание обобщений может привести к созданию неограниченно расширяющегося дерева конфигураций. В то же время, излишне энергичное применение обобщений сокращает дерево до размеров, при которых оно утрачивает свою ценность.

Из теоремы Хигмана-Крускала следует, что если из исходной последовательности выбрать любую бесконечную подпоследовательность, то ситуация вложения возникнет и для неё (ибо теорема приложима к любой бесконечной последовательности). Отсюда возникает идея разбить множество всех возможных конфигураций на непересекающиеся подмножества и сравнивать только те конфигурации, которые принадлежат к одной и той же категории. Эта идея была реализована в суперкомпиляторе SPSC [14] и описана в статье [15]. SPSC разбивает все конфигурации на три категории: «тривиальные», «глобальные» и «локальные».

- T - тривиальные
 - Параметр: v
- G - глобальные (редукция с сужением)
 - $g2(g1(v, \dots), \dots)$
- L - локальные (редукция без сужения)
 - $g2(g1(C(\dots), \dots), \dots)$
 - $g2(g1(f(\dots), \dots), \dots)$
 - $f(\dots)$

Тривиальные конфигурации обладают тем свойством, что любая последовательность, состоящая только из тривиальных конфигураций, — конечна. Поэтому тривиальные конфигурации можно вообще игнорировать при сравнении конфигу-

раций. Локальные конфигурации соответствуют шагам прогонки, которые можно выполнить, не используя информацию о значениях параметров, а глобальные конфигурации соответствуют шагам прогонки, при выполнении которых требуется анализ значений параметров (путем разбора случаев).

Разделение «локального» и «глобального» управления заикливанием заключается в том, что локальные конфигурации сравниваются только с локальными, а глобальные — с глобальными.

Метод разделения конфигураций на непересекающиеся множества позволяют суперкомпилятору производить более тонкий анализ программы, при этом он гарантирует конечность графа конфигураций.

5.3 Наиболее тесное обобщение

Обобщением конфигураций C_1 и C_2 называют кортеж $\langle C^*, \sigma_1, \sigma_2 \rangle$ (C^* — конфигурация, σ_1, σ_2 — подстановки), такой что $C^*/\sigma_1 = C_1$ и $C^*/\sigma_2 = C_2$.

Наиболее тесным обобщением конфигураций C_1 и C_2 называют обобщение $\langle \hat{C}, \sigma_1, \sigma_2 \rangle$, такое что для любого обобщения C' существует подстановка σ' , такая что $C'/\sigma' = \hat{C}$.

Опишем алгоритм нахождения наиболее тесного обобщения. Пусть даны конфигурации C_1 и C_2 , тогда начнём с тривиального обобщения $\langle x, \{x \rightarrow C_1\}, \{x \rightarrow C_2\} \rangle$, и пока происходят изменения будем выполнять операции:

1. Выделение общей функции/конструктора.

$$\begin{aligned} &\langle e, \{\dots, v \rightarrow F(e'_1, \dots, e'_n), \dots\}, \{\dots, v \rightarrow F(e''_1, \dots, e''_n), \dots\} \rangle \\ &\quad \Downarrow \\ &\langle e/\{v \rightarrow F(v_1, \dots, v_n)\}, \{\dots, v_1 \rightarrow e'_1, \dots, v_n \rightarrow e'_n, \dots\}, \\ &\quad \{\dots, v_1 \rightarrow e''_1, \dots, v_n \rightarrow e''_n, \dots\} \rangle \end{aligned}$$

2. Слияние параметров.

$$\begin{aligned} &\langle e, \{\dots, v_1 \rightarrow e', \dots, v_2 \rightarrow e'), \dots\}, \{\dots, v_1 \rightarrow e'', \dots, v_2 \rightarrow e''), \dots\} \rangle \\ &\quad \Downarrow \\ &\langle e/\{v_1 \rightarrow v_2\}, \{\dots, v_2 \rightarrow e', \dots\}, \{\dots, v_2 \rightarrow e'', \dots\} \rangle \end{aligned}$$

5.4 Прогонка

Более подробно опишем процесс прогонки. Пусть необходимо прогнать конфигурацию $f(E_1, \dots, E_m)$, где f — вызов функции, а $E_1, \dots, E_m$ — некоторые подвыражения. Будем перебирать правила редукции $f(P_1^{(k)}, \dots, P_m^{(k)}) \rightarrow X^{(k)}$ и решать для каждого правила следующую систему уравнений:

$$\begin{cases} E_1 : P_1^{(k)} \\ E_2 : P_2^{(k)} \\ \dots \\ E_m : P_m^{(k)} \end{cases}$$

Если система совместна, то для конфигурации $f(E_1, \dots, E_m)$ и k -го правила редукции $f(P_1^{(k)}, \dots, P_m^{(k)}) \rightarrow X^{(k)}$ существует сужение $\rho^{(k)}$ и подстановка $\sigma^{(k)}$, такие что $f(E_1/\rho^{(k)}, \dots, E_m/\rho^{(k)})$ редуцируется в $X^{(k)}/\sigma^{(k)}$.

Таким образом, для каждой совместной системы мы получим новую конфигурацию $X^{(k)}/\sigma^{(k)}$, являющуюся результатом редукции исходной конфигурации при сужении $\rho^{(k)}$. Новую конфигурацию следует подвесить к исходной, ассоциируя с ребром найденное сужение.

Чтобы решить уравнение $E : P$ необходимо следовать правилам:

$$1. C(E_1, \dots, E_m) : C(P_1, \dots, P_m) \Rightarrow \begin{cases} E_1 : P_1 \\ \dots \\ E_m : P_m \end{cases},$$

где C — конструктор, $E_1, \dots, E_m$ — некоторые подвыражения конфигурации, $P_1, \dots, P_m$ — шаблоны аргументов.

$$2. par : P \Rightarrow \begin{cases} par \rightarrow P' \\ par_1 \leftarrow var_1 \\ \dots \\ par_m \leftarrow var_m \end{cases},$$

где par — параметр, P' — это P с заменой переменных на новые параметры конфигураций; $par_1, \dots, par_m$ — новые параметры; $var_1, \dots, var_m$ — переменные шаблона P ; стрелка вправо означает сужение, а стрелка влево — подстановку.

$$3. E : var \Rightarrow E \leftarrow var.$$

4. $g(E_1, \dots, E_m) : C(P_1, \dots, P_m)$ — сопоставление невозможно, необходимо прогнать g .
5. $C_1(E_1, \dots, E_m) : C_2(P_1, \dots, P_m)$ — сопоставление невозможно, решений нет.

5.5 Основные классы и модули

Лексический и синтаксический анализ реализован в суперкомпиляторе с помощью библиотеки `ruparsing`. Она позволяет легко строить гибкие и эффективные синтаксические анализаторы и не требует никаких дополнительных утилит.

Внутри суперкомпилятора программа представляется как набор правил редукции в виде экземпляра класса `Program`; `RuleN` — класс правил; `Exp`, `Var`, `Call`, `Let`, `Ctr` — классы для представления выражений и конфигураций. У выражений реализован важный метод `applySubst(subst)`, позволяющий применять подстановку.

Узлы графа конфигураций представляются экземплярами класса `Node`. Ключевые методы класса `Node` приведены в таблице 1.

Таблица 1 — Методы класса `Node`

Метод	Предназначение
<code>funcAncestors()</code>	Поиск эквивалентной, включая имена переменных, конфигурации среди предков
<code>isProcessed()</code>	Проверка на необходимость обработки узла
<code>subtreeLeaves()</code>	Итерирование по всем листовым узлам в поддереве

Для представления дерева конфигураций используется класс `ProcessTree`. Основные методы этого класса приведены в таблице 2.

Таблица 2 — Методы класса `ProcessTree`

Метод	Предназначение
<code>addChildren(node, branches)</code>	Добавление к узлу <code>node</code> новых ветвей
<code>replaceSubtree(node, exp)</code>	Замена поддерева выражения конфигурации <code>node</code> на <code>exp</code> с удалением всех потомков

Метод `addChildren(node, branches)` вызывается после прогонки конфигурации, чтобы добавить возможные пути выполнения программы в дерево процессов.

Модуль `he.py` реализует функцию `embeddedIn(e1, e2)` проверки гомеоморфного вложения конфигурации `e1` в конфигурацию `e2`.

Модуль `matcher.py` предоставляет инструменты для сопоставления шаблонов и выражений, проверки вложенности¹ и эквивалентности конфигураций.

В модуле `msg.py` реализован алгоритм построения наиболее тесного обобщения конфигураций.

Для осуществления прогонки конфигураций суперкомпилятор решает систему уравнений, у которой в левой части стоит выражение, а в правой — шаблон. Решение осуществляется с помощью класса `Solver`.

Предназначение класса `ProcessTreeBuilder` – построение графа конфигураций. В таблице 3 (приложение А) описываются методы этого класса.

¹В смысле частного случая.

6 Проектирование усовершенствованного суперкомпилятора

Пусть для некоторой конфигурации C с множеством параметров $dom(C)$ имеется гипотеза выходного формата FMT_C с множеством неизвестных $dom(FMT_C)$, тогда будем считать, что конфигурация $C : FMT_C$ вычисляет такую подстановку σ в FMT_C , что FMT_C/σ описывает в точности $RANGE(C)$. Причем необходимая подстановка определяется по мере суперкомпиляции $C : FMT_C$ – подграфа.

Теперь будем строить вычислительный граф, в котором каждая конфигурация имеет вид $C : FMT_C$. Для новых конфигураций будем выдвигать нулевую гипотезу $\perp$ (дно стека), означающую, что процесс вычислений на этой ветви никогда не останавливается. Формат прогоняемой конфигурации наследуется потомками. Однако в случае если по ходу суперкомпиляции унаследованная гипотеза опровергнется, то это повлияет сразу на всех обладателей гипотезы вплоть до прародителя, и весь этот подграф целиком будет перестроен вновь с новой, уточненной гипотезой. В разделе 6.3 описан способ проверки форматных гипотез.

6.1 Создание let-узлов

Основной целью создания let-узла во время построения вычислительного графа является обобщение — отказ от дополнительной информации о состоянии вычислительного процесса в пользу конечности графа конфигураций (т.е. в пользу завершаемости самого процесса суперкомпиляции). Кроме того, let-узлы создаются с целью декомпозиции внешнего конструктора. В этом случае никакая информация не теряется, поскольку конструктор, «всплывший» наружу в вычислениях больше не участвует.

Пусть имеется узел $\text{let } (\{x := E_x, y := E_y, \dots\} \text{ in } V) : FMT_V$. Но в нашем вычислительном графе каждая конфигурация должна иметь гипотезу о выходном формате. Это правило распространяется и на конфигурации $E_x, E_y, \dots$. Эти конфигурации независимые, они не наследуют формат, и для каждой из них суперкомпилятор заведёт свою гипотезу. По итогу анализа let-ветвей мы получим подграф, изображенный на рисунке 1.

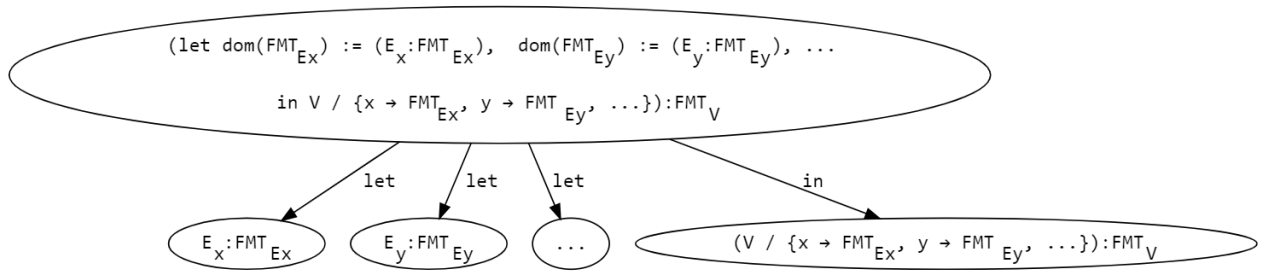


Рисунок 1 — let-узел

Предлагаемый алгоритм находит еще одно применение let-узлам: пусть C_f — конфигурация с форматом FMT_{C_f} (гипотеза), C — некоторая конфигурация, лежащая ниже в поддереве относительно C_f , причём C содержит частный случай C_f , или более формально — $C = C' / \{x \rightarrow C_f / \sigma\}$. Назовём такое отношение «обобщённым вложением».¹, тогда создадим узел $(let\ dom(FMT_{C_f}) := (C_f / \sigma) : FMT_{C_f} \text{ in } C' / \{x \rightarrow FMT_{C_f}\}) : FMT_C$ (рисунок 2). По ветке let естественным образом возникает еще один узел $(let\ \sigma \text{ in } C_f) : FMT_{C_f}$. Этот узел обрабатывается так, как это описано в предыдущем абзаце: элементы подстановки по let-ветвям разбиваются на конфигурации, конфигурации получают форматную гипотезу и суперкомпилируются.

Отметим также два важных свойства форматов. Первое — если в конце суперкомпиляции у некоторой конфигурации имеется нетривиальный выходной формат, то это означает, что при любых значениях параметров, приводящих к завершению процесса вычисления конфигурации (без ошибок), результат будет обязательно удовлетворять формату. Второе — если в конце суперкомпиляции у некоторой конфигурации сохранился тривиальный выходной формат, то это означает, что на любых значениях параметров процесс вычисления либо остановится с ошибкой, либо никогда не завершится.

6.2 Обработка let-узлов

Необходимо разделять let-узлы по типу:

¹Обобщённое вложение конфигурации C_f в C похоже на гомеоморфное вложение, однако в общем случае неверно, что $C_f \trianglelefteq C$ (т.е. C_f гомеоморфно вложено в C). Контрпример: $C_f = f(x)$, $C = f(Z)$. В этом случае $C_f \not\trianglelefteq C$, однако $C = y / \{y \rightarrow f(x) / \{x \rightarrow Z\}\}$ — C_f обобщённо вложено в C .

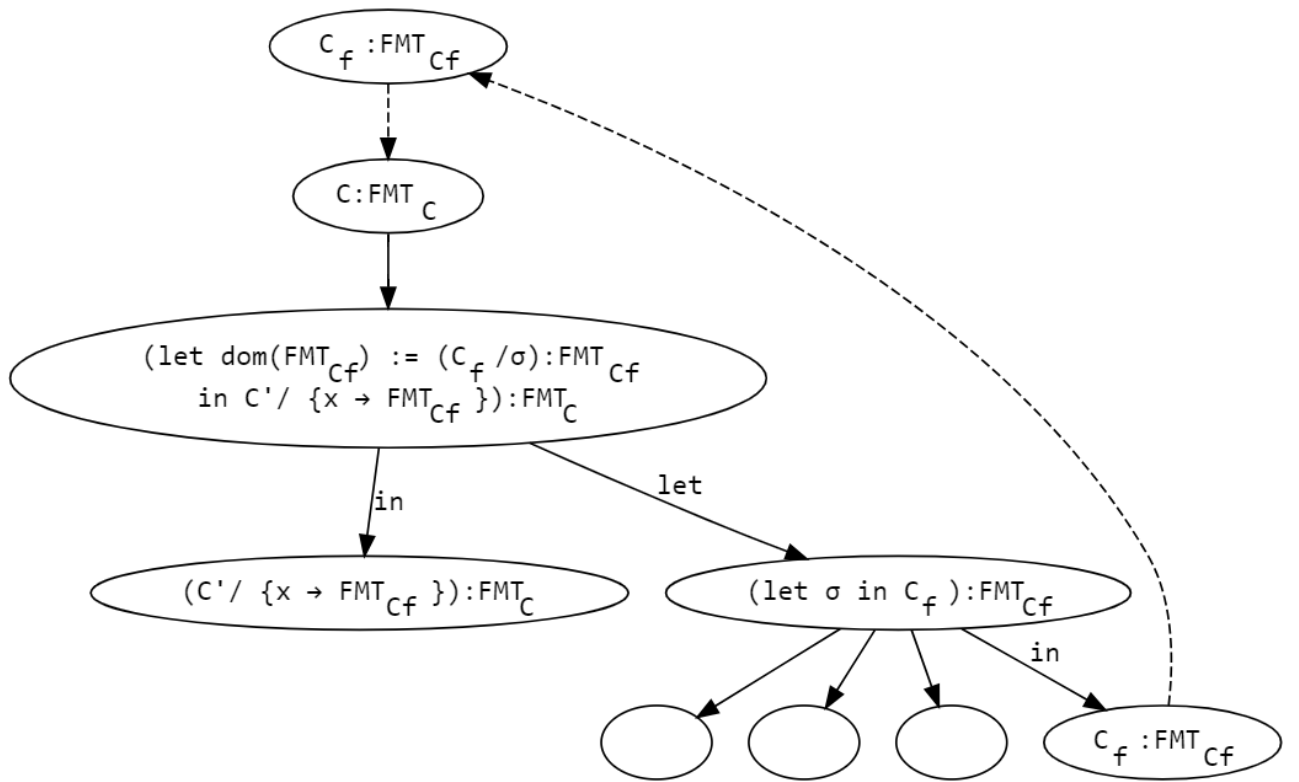


Рисунок 2 — let-узлы, обрабатывающие обобщённое вложение

1. SPECIAL_CASE — создаётся в случае, если среди конфигураций обнаружено вложение в смысле частного случая.
2. HE_ABSTRACT — необходим для обработки гомеоморфного вложения, когда наиболее тесное обобщение нетривиально. Этот let-узел заменяет собой конфигурацию, лежащую выше по дереву.
3. HE_SPLIT — используется для обработки гомеоморфного вложения, в случае когда наиболее тесное обобщение получилось тривиальным. Заменяет собой конфигурацию, лежащую ниже по дереву.
4. CTR_DECOMPOSE — необходим для декомпозиции конструкторов.
5. GENERAL_EMB — узел, обслуживающий случай обобщённого вложения.

При обработке let-узлов сначала необходимо просуперкомпилировать let-ветви, а потом уже разбираться с ветвью in. Предположим, что с первым этапом мы успешно справились, и теперь у нас есть выходные форматы каждого let-поддерева. В общем случае нам бы хотелось подставлять выходные форматы этих let-ветвей в тело let-узла, в ветку in, чтобы это в конце концов повлияло и на его выходной формат. Оказывается, сделать это можно далеко не всегда. Например, сомнительной кажется идея осуществлять эту подстановку в let-узел типа

SPECIAL_CASE, поскольку в этом случае по ветке `in` пропадёт заикливание. Подстановка формата в тело `let`-узла типа `HE_ABSTRACT` лишает смысла обобщение и может привести к заикливанию процесса суперкомпиляции¹.

Напротив, узел типа `HE_SPLIT` допускает подстановку формата в `in`, поскольку ветка `in` не участвует в разрешении опасного вложения. А идея `let`-узла типа `GENERAL_EMB` как раз и заключается в подстановке формата единственной `let`-ветви в тело. И наконец последний тип, `CTR_DECOMPOSE`, также допускает подстановку.

Без внимания остался тот факт, что среди форматных гипотез может встретиться $\perp$ (дно стека). И после подстановки такой гипотезы в выражение конфигурации возникнет незнакомая для суперкомпилятора ситуация (действительно, как вычислить, например, выражение $f(\perp)$, где f – различающая внешний конструктор аргумента функция?). Можно ли вообще отказаться от такой подстановки? Да, но тогда во многих случаях придется пожертвовать точностью форматной гипотезы². Чтобы избежать подобной ситуации, будем осуществлять подстановку только в `let`-узлах типа `CTR_DECOMPOSE`. Это ограничение гарантирует нам отсутствие активных выражений, содержащих в себе $\perp$. Кроме того, пассивное выражение, содержащее $\perp$ можно отождествлять с выражением $\perp$, и считать, что оно удовлетворяет любой гипотезе, или по крайней мере никакую гипотезу не опровергает.

6.3 Проверка форматных гипотез

Пусть C – пассивное выражение, FMT_C – ассоциированный с ним выходной формат. Для таких конфигураций выполняется обобщение гипотезы и выражения по следующему алгоритму:

- Если $FMT_C = \perp$, тогда:
 - Если C содержит $\perp$, то возвращается $\perp$.
 - Если C не содержит $\perp$, то возвращается C .
- Иначе возвращается $MSG(C, FMT_C)$.

¹Например $f(Z) \rightarrow f(S(Z))$ – опасное вложение, которое разрешится заменой $f(Z)$ на `let x:=Z in f(x)`, и если подставить формат Z в тело, получим опять $f(Z)$, и процесс суперкомпиляции никогда не остановится.

²Например, суперкомпиляция программы `sum_3x.scala` (см. раздел 8) без подстановки формата $\perp$ в тело `let`-узла типа `CTR_DECOMPOSE` приведет к извлечению «наружу» только одного конструктора S .

Если результат обобщения совпадает с FMT_C с точностью до переименования параметров, то гипотеза сохраняется, иначе – меняется на полученный результат.

6.4 Генерация остаточной программы

Использование гомеоморфного вложения гарантирует конечность графа конфигураций, а это значит, что по полученному графу можно построить программу конечного размера. Необходимость модифицировать классический алгоритм генерации остаточной программы обусловлена следующим различием – конфигурация теперь вычисляет подстановку в формат, а не пассивное выражение, содержащее, быть может, вызовы остаточных функций.

Как и прежде, каждый базисный узел в графе помечается неким идентификатором, который будет служить именем остаточной функции; параметры, встречающиеся в конфигурации, становятся аргументами функции; сужения, полученные в ходе прогонки, преобразовываются в правила редукции новой программы. Однако, остаточные функции теперь могут возвращать несколько значений, поэтому необходимо расширить выходной язык относительно модельного, добавив также вызовы с присвоением.

Остаточная программа строится рекурсивно снизу вверх. Листовыми узлами вычислительного графа могут быть конфигурации с пассивными выражениями, либо конфигурации с обратными стрелками. Сопоставление формата с пассивной конфигурацией даёт необходимую подстановку — результат вычисления всего подграфа. В случае «зацикленных» конфигураций используется приём из обобщённого вложения — на место выражений подставляются выходные форматы конфигураций, на которые указывают обратные стрелки. В результате сравнения форматов получается подстановка, которая вкупе с вызовом соответствующей остаточной функции даст необходимый результат.

Если в каких-то let-ветвях встречается прогоняемые конфигурации, то чтобы избежать конфликтов сужений параметров, для этих let-ветвей заводятся остаточные функции.

7 Реализация

Для выполнения задания был составлен план реализации нового функционала и логики суперкомпиляции:

1. Добавить возможность графического отображения дерева конфигураций.
2. Заменить основной цикл суперкомпиляции на рекурсивную функцию.
3. Реализовать функцию поиска обобщённого вложения¹.
4. Реализовать алгебру обобщения и сопоставления выражения конфигурации с форматной гипотезой.
5. Обеспечить подстановку выходных форматов в `in`-выражения `let`-узлов.
6. Реализовать алгоритм генерации остаточной программы

7.1 Графическое отображение дерева конфигураций

Отсутствие возможности графического отображения дерева конфигураций в прошлой версии суперкомпилятора затрудняло отладку и подготовку отчёта о работе. Для решения этой проблемы был использован инструмент Graphviz.

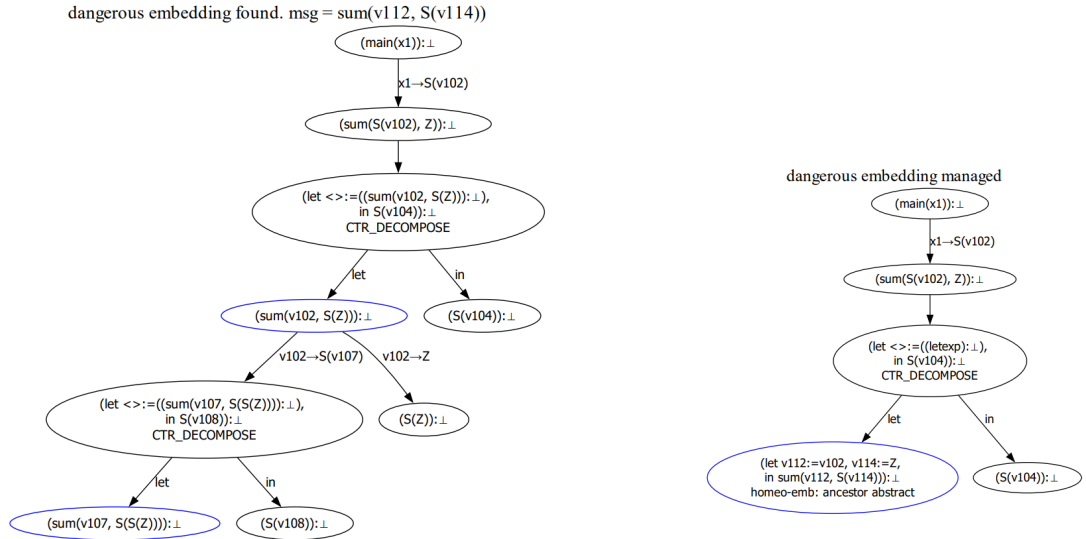
Класс `ProcessTree` пополнился новым методом `render(title, nodesToFocus, focusColor)`, который предоставляет необходимый функционал для отображения текущего состояния графа конфигураций. Результаты отрисовки сохраняются в папку `progress/` в виде изображений, которые в конце объединяются в один pdf-файл. Это позволяет, при необходимости, получить подробный отчет о всех этапах построения дерева конфигураций, что значительно облегчает отладку и интерпретацию результатов суперкомпиляции.

На рисунках 3 а) и б) изображён промежуточный этап построения графа конфигураций. По подписи сверху указывают, что на данном этапе было обнаружено, а потом обработано опасное вложение.

7.2 Преобразования основного цикла в рекурсивную функцию

Процесс построения дерева конфигураций был организован как цикл по необработанным листовым вершинам. Но выяснилось, что такой подход плохо

¹Выражение C_f обобщённо вложено в C , если $C = C' / \{x \rightarrow C_f / \sigma\}$



а) Обнаружено опасное вложение

б) Опасное вложение обработано

Рисунок 3 — Визуализация графа конфигураций в процессе построения

подходит для внедрения логики построения выходных форматов, ввиду необходимости соблюдения определенного порядка суперкомпиляции вершин, подвешенных к let-узлам. Этот порядок легко соблюдается, если заменить цикл на рекурсивную функцию суперкомпиляции.

Функция `buildProceesTree(k)`, реализующая основной цикл суперкомпиляции, была заменена на рекурсивную функцию `buildRecursive(beta)`. Функция `buildRecursive(beta)` — сердце суперкомпиляции. Здесь сосредоточена основная логика. Рассмотрим её реализацию.

Итак, на вход подаётся `beta` — член класса `Node`. В первую очередь проверяется пассивно ли выражение узла. Если выражение пассивно и в нём встречается $\perp$ (ответ на вопрос, как в конфигурациях может появляться дно стека, дан в разделе 6.2), то это означает, что выражение можно отождествить с $\perp$. Поскольку выражение пассивно, далее ожидается проверка на соответствие с гипотезой. Но $\perp$ удовлетворяет любым гипотезам, поэтому далее следует возврат из рекурсивной функции (листинг 8).

Если выражение пассивно и его выходной формат — дно стека, гипотеза отвергается в пользу выражения узла. Далее все поддереву формата перестраивается (листинг 8).

Листинг 8: Функция построения дерева конфигураций (1/7)

```
1 def buildRecursive(self, beta):
2     if beta.isPassive():
3         if beta.exp.hasStackBottom():
4             return
5         if beta.outFormat.exp.isStackBottom():
6             self.tree.render("Hypothesis is refuted", [beta], focusColor='red')
7             newfmt = copy.deepcopy(beta.exp)
8             newfmt.changeVarsToNewParams(self.nameGen)
9             root = beta.outFormat.root
10            root.outFormat.exp = newfmt
11            root.children = []
12            self.tree.render("New format hypothesis", [root])
13            self.buildRecursive(root)
14        else:
15            mg = matchAgainst(beta.outFormat.exp, beta.exp)
16            if mg is None:
17                gen = self.msgBuilder.build(beta.outFormat.exp, beta.exp)
18                self.tree.render("Hypothesis is refuted", [beta], focusColor='red')
19                root = beta.outFormat.root
20                root.outFormat.exp = gen.exp
21                root.children = []
22                self.tree.render("New format hypothesis", [root])
23            self.buildRecursive(root)
24    return
```

Если выражение пассивно и его выходной формат не дно стека, то сначала проверяется, что выражение является частным случаем выходного формата, и если это не выполняется, то гипотеза отвергается в пользу наиболее тесного обобщения выражения и формата. Далее все поддереву формата перестраивается (листинг 8).

С пассивными выражениями разобрались. Обработка `let`-узлов, будет рассмотрена отдельно (см. раздел 6.2). Перейдём к рассмотрению активных выражений.

Если перед нами конструктор, то необходимо заменить выражение на `let`-узел типа `CTR_DECOMPOSE`¹ и обработать его (листинг 9).

Если для выражения существует предок `alpha` в точности совпадающий с `beta`, то из `beta` будет идти обратная стрелка в `alpha`, а наша задача здесь проверить гипотезу узла `beta`, подставив выходной формат `alpha` вместо выражения (листинг 10).

¹О типах `let`-узлов см. в разделе 6.2

Листинг 9: Функция построения дерева конфигураций (2/7)

```
1  if beta.exp.isCtr():
2      e = beta.exp
3      names = self.nameGen.freshNameList(len(e.args))
4      args = [Var(x) for x in names]
5      letExp = Let(e.cloneFunctor(args), list(zip(names, e.args)), Let.Type.CTR_DECOMPOSE)
6      self.tree.replaceSubtree(beta, letExp)
7      self.manageLet(beta)
8      return
```

Листинг 10: Функция построения дерева конфигураций (3/7)

```
1  if beta.isProcessed():
2      alpha = beta.funcAncestor()
3      if not alpha.outFormat.exp.isStackBottom():
4          if beta.outFormat.exp.isStackBottom() or not instOf(alpha.outFormat.exp,
5              ↪ beta.outFormat.exp):
6              self.tree.render("Hypothesis is refuted", [beta], focusColor='red')
7              newfmt = copy.deepcopy(alpha.outFormat.exp)
8              newfmt.changeVarsToNewParams(self.nameGen)
9              root = beta.outFormat.root
10             root.outFormat.exp = newfmt
11             root.children = []
12             self.tree.render("New format hypothesis", [root])
13             self.buildRecursive(root)
14
15     return
```

Если beta является частным случаем некоторого предка alpha, найдём соответствующую подстановку, заменим beta на let-узел типа SPECIAL_CASE и обработаем его (листинг 11).

Листинг 11: Функция построения дерева конфигураций (4/7)

```
1  if beta.exp.isFGHCall():
2      moreGenAnc = beta.findMoreGeneralAncestor() #FGHCall only
3      if moreGenAnc:
4          alpha = moreGenAnc
5          self.tree.render("Special case found", [beta, alpha])
6          subst = matchAgainst(alpha.exp, beta.exp)
7          bindings = list(subst.items())
8          bindings.sort()
9          letExp = Let(alpha.exp, bindings, Let.Type.SPECIAL_CASE)
10         self.tree.replaceSubtree(beta, letExp)
11         self.manageLet(beta)
12         return
```

Если в beta обнаруживается частный случай некоторого предка alpha, т.е. $\beta = \beta' / \{x \rightarrow \alpha / \sigma\}$, заменяем beta на let-узел типа GENERAL_EMB с телом β' и обрабатываем его (листинг 12).

Листинг 12: Функция построения дерева конфигураций (5/7)

```
1  if beta.exp.isFGHCall():
2      freshName = self.nameGen.freshName()
3      Cf, C, Cz = beta.findMoreGeneralEmbeddedAncestor(freshName) #FGHCall only
4      if Cf:
5          self.tree.render("More General Embedded Ancestor found", [beta, Cf])
6          letExp = Let(Cz, [(freshName, C)], Let.Type.GENERAL_EMB)
7          self.tree.replaceSubtree(beta, letExp)
8          self.tree.render("Let created", [beta, Cf])
9          self.manageLet(beta)
10         return
```

Если некоторый предок α гомеоморфно вложен в β , тогда находим наиболее тесное обобщение $\text{gen} = \text{msg}(\alpha, \beta)$, и если gen – параметр, то выполняем декомпозицию функции β , вводя let -узел типа HE_SPLIT , а если обобщение нетривиально, то заменяем α на let -узел типа HE_ABSTRACT с телом gen и с let -подстановкой σ_α такой, что $\text{gen}/\sigma_\alpha = \alpha$ (листинг 23). Функции `abstract` и `split` приведены в листинге 25.

Если ни одно из вышеперечисленных условий не выполнилось, то перед нами активное выражение с внешним вызовом функции, требующее прогонки (листинг 24).

7.3 Реализация функции поиска обобщённого вложения

Алгоритм, проверяющий является ли одна конфигурация частным случаем другой, задаётся набором рекурсивных правил:

1. Любое выражение E является частным случаем параметра x .
2. $f(X_1, \dots, X_k)$ частный случай $f(Y_1, \dots, Y_k)$, если f — конструктор или функция, и для всех i выполнено X_i — частный случай Y_i .

Тогда отношение обобщенного вложения между двумя выражениями задаётся правилами:

1. Параметр x обобщенно вложен в E для любого выражения E .
2. Выражение E обобщенно вложено в $f(Y_1, \dots, Y_k)$, если f — имя конструктора или функции и E обобщенно вложено в Y_i для некоторого i .
3. $f(X_1, \dots, X_k)$ обобщенно вложено в $f(Y_1, \dots, Y_k)$, если f — конструктор или функция и для всех i выполнено Y_i — частный случай X_i .

В листинге 13 приведена реализация алгоритма проверки наличия обобщенного вложения.

Листинг 13: Реализация алгоритма проверки наличия обобщённого вложения

```
1 def moreGeneralEmbeddedIn(e1, e2, newName):
2     return ge(e1, e2, newName)
3
4 def ge(e1, e2, newName):
5     return (e2, Var(newName)) if geByCoupling(e1, e2) else geByDiving(e1, e2, newName)
6
7 def geByDiving(e1, e2, newName) :
8     if e2.isVar():
9         return (None, None)
10    elif e2.isCall():
11        for i, e in enumerate(e2.args):
12            (C, E) = ge(e1, e, newName)
13            if C:
14                newE = e2.cloneFunctor(e2.args)
15                newE.args[i] = E
16                return (C, newE)
17        return (None, None)
18
19 def geByCoupling(e1, e2):
20     if e1.isVar():
21         return True
22     elif e1.hasTheSameFunctorAs(e2):
23         return all(map(geByCoupling, e1.args, e2.args))
```

7.4 Реализация генератора остаточной программы

Генератор остаточной программы по дереву конфигураций получает case-правила остаточных функций. Эти правила функций нового расширенного выходного языка описываются классом `ARule` в модуле `language.py`. Помимо привычных членов `name` и `patternList` класс обладает полями `letCallList` и `bodyList`, которые содержат вызовы с присвоением и список возвращаемых выражений соответственно.

Сразу после построения вычислительного графа, узлы, принимающие обратные стрелки, помечаются флагом `isBase`. Они будут преобразованы генератором в остаточные функции. Рекурсивная функция построения остаточной программы `defineFormatParams(node)` из класса `advancedResidualProgramGenerator` (листинг 14) во время обхода вычислительного графа в глубину накапливает информацию о будущих правилах остаточной функции, возвращая всякий раз список экземпляров класса `ProtoRule`. Размер этого списка соответствует числу различных сужений, встретившихся по пу-

ти. Сами же экземпляры содержат поля `contr` – сужения параметров, `letCallList` и `bodyList` – вызовы с присвоением и список возвращаемых выражений соответственно. В тот момент когда функция `defineFormatParams` добирается до базисной конфигурации, создаётся новая остаточная функция и накопленная информация превращается в её правила `ARule`. Корневые вершины `let`-ветвей также «выпадают в осадок», но только в случае, если ниже по поддереву встречалась прогонка.

Листинг 14: Рекурсивная функция генератора остаточной программы

```
def defineFormatParams(self, node):
    while len(node.children) == 1 and not node.children[0].contr and not node.exp.isPassive()
    ↪ and not node.exp.isLet() and node.isBase == Node.IsBase.FALSE:
        node.protos = copy.deepcopy(self.defineFormatParams(node.children[0]))
        return node.protos

    if node.exp.isPassive():
        subst = matchAgainst(node.outFormat.exp, node.exp)
        node.protos = [ProtoRule(bodyList=[subst[vname] for vname in
    ↪ node.outFormat.exp.vars()])] if subst else [ProtoRule()]
        return node.protos

    if not node.exp.isLet():
        resProtos = self._handleNonLetNode(node)
        node.protos = resProtos
        if node.isBase in {Node.IsBase.TRUE, Node.IsBase.CANDIDATE} and (node.isBase ==
    ↪ Node.IsBase.TRUE or len(resProtos) > 1):
            self.genAFunc(node)
            sig = self.sigs[node]
            node.protos = [ProtoRule(bodyList=[HCall(sig[0], [Var(vname) for vname in
    ↪ sig[1]])])]
            return node.protos

    return self._handleLetNode(node)
```

Реализация вспомогательных функций генератора приведена в листингах 15 - 16. На первый взгляд, логика генератора, содержащая большое количество разнообразных проверок, особенно при обработке `let`-узлов, кажется слишком запутанной и громоздкой. Однако такая витиеватость возникает ввиду большого числа различных возможных ситуаций, а также желания избежать выпадения в осадок транзитных узлов и появления избыточных присвоений. Действительно, одних только `let`-узлов встречается пять типов, и к каждому нужен особый подход. Помимо этого, каждая конфигурация (включая `let`-узлы) имеет один из трёх возможных типов выходных форматов: параметризованный, непараметризованный и дно стека.

Листинг 15: Вспомогательная функция генератора (1)

```
def _handleLetNode(self, node):
    assert node.exp.isLet()

    if node.children[-1].isBase != Node.IsBase.TRUE:
        node.children[-1].isBase = Node.IsBase.CANDIDATE

    inProto = copy.deepcopy(self.defineFormatParams(node.children[-1])[0])
    for i, (let, (vname, _)) in enumerate(zip(node.children[:-1], node.exp.bindings)):
        if not node.children or node.children[i].exp.isVar() or not
        ↪ node.exp.can_insert_format_to_body() or
        ↪ (node.children[i].outFormat.exp.isStackBottom() and node.exp.type !=
        ↪ Let.Type.CTR_DECOMPOSE):
            if not node.children or node.children[i].outFormat.exp.isStackBottom() or
            ↪ node.children[i].exp.isVar():
                let.outFormat.exp = Var(vname)

    if let.isBase != Node.IsBase.TRUE:
        let.isBase = Node.IsBase.CANDIDATE

    protos = self.defineFormatParams(let)
    letBodyList = protos[0].bodyList
    callListToAdd = copy.deepcopy(protos[0].letCallList)
    inBodyList = inProto.bodyList

    letSubst = {vname: letBodyList[0] if not node.children or
    ↪ node.children[i].outFormat.exp.isStackBottom() or node.children[i].exp.isVar() else
    ↪ let.outFormat.exp}
    for j in range(len(inBodyList)):
        inBodyList[j] = inBodyList[j].applySubst(letSubst)
    for j in range(len(inProto.letCallList)):
        inProto.letCallList[j].call = inProto.letCallList[j].call.applySubst(letSubst)

    if not let.exp.isVar() and (not let.outFormat.exp.isStackBottom() or node.exp.type ==
    ↪ Let.Type.CTR_DECOMPOSE) and not (let.exp.isPassive() and not let.exp.hasVar()):
        if len(letBodyList) == 1:
            callListToAdd.append(LetCall(let.outFormat.exp.vars(), letBodyList[0]))
        else:
            for vname, letBody in zip(let.outFormat.exp.vars(), letBodyList):
                callListToAdd.append(LetCall([vname], letBody))

    inProto.letCallList = callListToAdd + inProto.letCallList

    node.protos = [inProto]
    return node.protos
```

Например, если некоторая `let`-ветвь, соответствующая замене параметра `v`, имеет форматную гипотезу $\perp$, то в случае `CTR_DECOMPOSE`, дно стека попадет в тело `let`-узла, а в `letCallList` добавится вызов остаточной функции `let`-ветви (которая, судя по формату, никогда не завершится). Для других типов `let`-узлов действовать нужно иначе — подстановка $\perp$ в тело не производится, а в `bodyList`

let-узла все вхождения параметра *v* будут заменены на остаточное выражение нашей let-ветви с подменённой на переменную гипотезой¹.

Листинг 16: Вспомогательная функция генератора (2)

```
def _handleNonLetNode(self, node):
    if node.isBase == Node.IsBase.TRUE:
        params = node.exp.vars()
        sig = self.getSig(node, node.exp.name, params)

    alpha = node.funcAncestor()
    if alpha:
        sig = self.sigs[alpha]
        if not equiv(node.outFormat.exp, alpha.outFormat.exp):
            subst = matchAgainst(node.outFormat.exp, alpha.outFormat.exp)
            letCall = LetCall(alpha.outFormat.exp.vars(), HCall(sig[0], [Var(vname) for vname in
                ↪ sig[1]]))
            bodyList = [subst[vname] for vname in node.outFormat.exp.vars()]
            return [ProtoRule(letCallList=[letCall], bodyList=bodyList)]
        else:
            return [ProtoRule(bodyList=[HCall(sig[0], [Var(vname) for vname in sig[1]])])]
    else:
        resProtos = []
        for child in node.children:
            protos = copy.deepcopy(self.defineFormatParams(child))
            for proto in protos:
                newContr = {key: value.applySubst(proto.contr) for key, value in
                    ↪ copy.deepcopy(child.contr).items()}
                newContr.update({key: value for key, value in proto.contr.items() if key not in
                    ↪ newContr})
                proto.contr = newContr
                resProtos.append(proto)
        return resProtos
```

Последний этап синтеза остаточной программы заключается в превращении правил ARules в case-предложения остаточных функций. Эта важная, но наименее интересная часть генератора, носит целиком технический характер и подробно рассмотрена не будет. Стоит лишь отметить, что программа на модельном языке, исходная или остаточная, не может быть сразу скомпилирована и выполнена с помощью Scala. Она всё ещё является корректной программой на Scala, но для её запуска необходимо добавить метод `main(args: Array[String]): Unit`. Это стандартный метод входа в программу на Scala, аналогичный `public static void main(String[] args)` в Java. Поэтому генератор имеет дополнительный режим работы, необходимый для автоматического тестирования (см. раздел 8), который позволяет генерировать остаточные функции вне контекста, из которых потом собирается тестовая программа.

¹Трюк с заменой осуществляется перед вызовом `defineFormatParams(letBranchNode)`, чтобы симитировать для let-ветви классический алгоритм нахождения остаточного выражения

8 Тестирование

Для тестирования суперкомпилятора был разработан модуль `supercompiler_tests.py`. Он включает в себя серию тестовых программ, которые преобразовываются в остаточные программы под действием суперкомпилятора, после чего корректность преобразования устанавливается проверкой на эквивалентность программ на конечном подмножестве области определения. Для этого программы сначала компилируются, потом запускаются на общих входных данных, а результат выполнения сравнивается на равенство.

Запустим суперкомпилятор на программе `sum_3x.scala` (листинг 17). Похожая программа была рассмотрена А.П.Немытых в работе «Суперкомпилятор SCP4: Общая структура» [5] в главе посвященной индуктивным выходным форматам. Используемый в SCP4 алгоритм смог построить формат лишь с одной единицей.

Листинг 17: Пример программы, увеличивающей число на три

```
case class S(x: Any)
case object Z

object Main {
  def main: Any => Any = {
    case n => sum(n, S(S(S(Z))))
  }
  def sum: (Any, Any) => Any = {
    case (S(n1), x) => S(sum(n1, x))
    case (Z, x) => x
  }
}
```

Полученный граф конфигураций изображён на рисунке 4. В ходе суперкомпиляции выходной формат корневой вершины менялся сначала с $\perp$ до $S(S(S(Z)))$, а потом с $S(S(S(Z)))$ до $S(S(S(v166)))$. В итоге, неучаствующие в вычислениях конструкторы «всплыли» наружу в выходной формат. В результате кодогенерации была получена программа (листинг 18), в которой функция `main` навешивает на результат вызова функции `sum1` три конструктора `S`. Сама же функция `sum1`, вопреки названию, сумму не вычисляет, и даже никак не меняет свой аргумент, однако она не лишена смысла, поскольку, перебирая параметры, она гарантирует, что остаточная программа, также как и исходная, завершится ошибкой в случае, если на вход будут переданы необъявленные конструкторы.

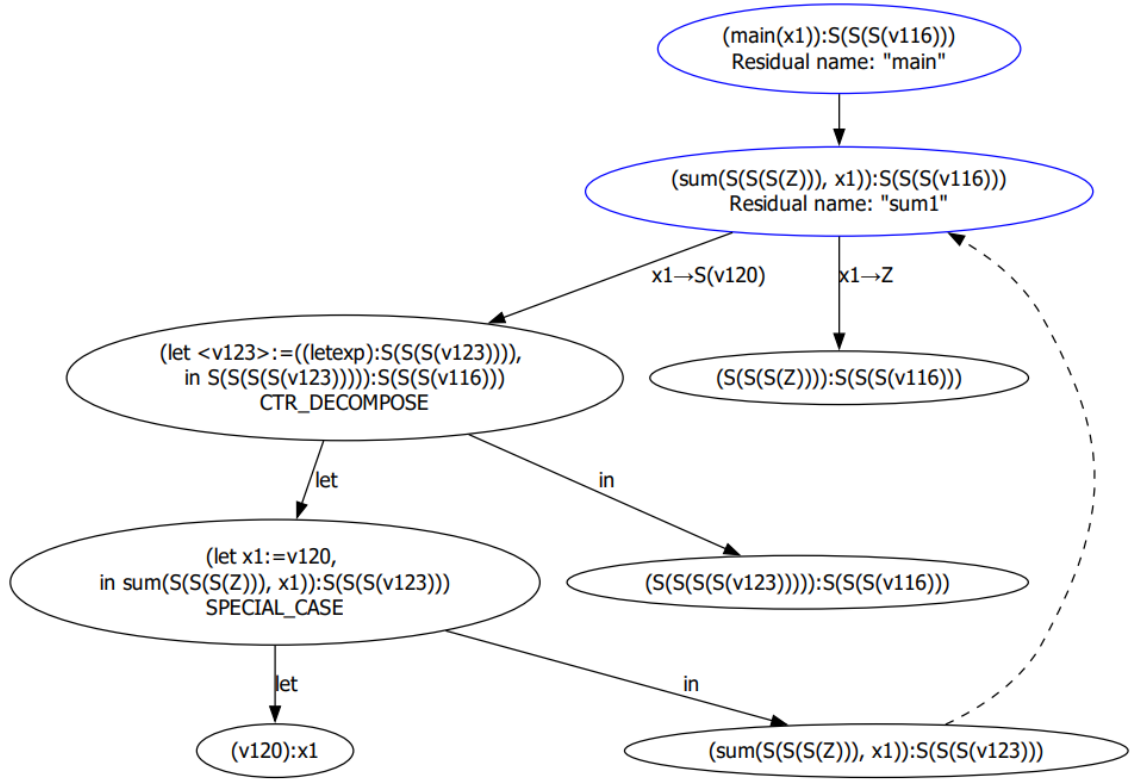


Рисунок 4 — Итоговый граф конфигурации для программы `sum_3x.scala`

Листинг 18: Результат суперкомпиляции программы `sum_3x.scala`

```

case class S(x1 : Any)
case object Z
object Main {
  def sum1 : Any => Any = {
    case S(v114) => S(sum1(v114))
    case Z => Z
  }
  def main : Any => Any = {
    case x1 => S(S(S(sum1(x1)))
  }
}

```

Рассмотрит программу `fac.scala` (листинг 26), реализующую функцию `fab`, которая преобразует все `A` в `B`, функцию `fbc`, переводящую все `B` в `C` и функцию `main`, являющуюся композицией `fbc` и `fab`, и при этом добавляющую `A` в начало списка. Очевидно, что под действием функций `fab` и `fbc` головной элемент списка изменится на `C`, а выходной формат будет `Cons(C, x)`. На рисунках 6 - 7 (приложение А) приведен финальных граф конфигурации. Легко видеть, что выходной формат корневой конфигурации соответствует ожидаемому. Остаточная программа представлена в приложении А в листинге 27.

Ещё одна программа `double_2+x.scala` (листинг 19) удваивает число, заведомо не меньшее единицы. Очевидно, что каким бы ни было входное число из области определения, результат всегда будет больше двух. Суперкомпилятор пришел к такому же выводу (рисунок 5). Отрисованные синим цветом узлы соответствуют функциям остаточной программы (листинг 20).

Листинг 19: Программа `double_2+x.scala`

```
case class S(x: Any)
case object Z

object Main {
  def main: Any => Any = {
    case S(x) => sum(S(x), Z)
  }
  def sum: (Any, Any) => Any = {
    case (S(x), y) => S(sum(x, S(y)))
    case (Z, y) => y
  }
}
```

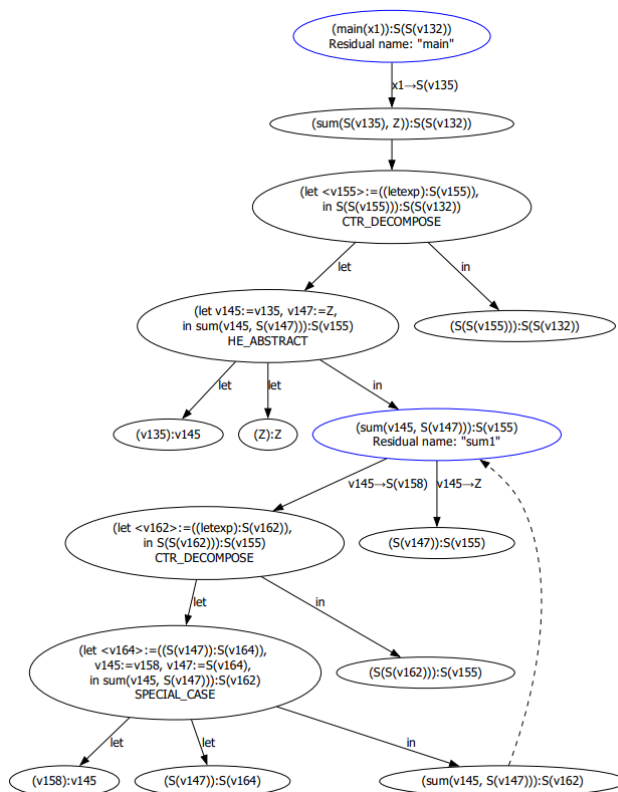


Рисунок 5 — Итоговый граф конфигурации для программы `double_2+x.scala`

Листинг 20: Результат суперкомпиляции программы double_2+x.scala

```
case class S(x1 : Any)
case object Z
object Main {
  def sum1 : (Any, Any) => Any = {
    case (S(v146), v137) => S(sum1(v146, S(v137)))
    case (Z, v137) => v137
  }
  def main : Any => Any = {
    case S(v128) => S(S(sum1(v128, Z)))
  }
}
```

До этого момента встречались примеры программ, у которых был только один параметр формата. Заполним этот пробел рассмотрением программы `sumpair.scala` (листинг 21), в которой, помимо `S` и `Z`, определён двухместный конструктор `Pair`. Функция `sumpair` принимает на вход два `Pair` и поэлементно их суммирует. Функция `main` своим единственным правилом налагает ограничение на входные данные, требуя чтобы элементы пар были не меньше единицы.

Листинг 21: Программа `sumpair.scala`

```
case class S(x: Any)
case object Z
case class Pair(x:Any, y:Any)

object Main {
  def main: (Any, Any) => Any = {
    case (Pair(S(x), S(y)), Pair(S(u), S(v))) => sumpair(Pair(S(x), S(y)), Pair(S(u), S(v)))
  }
  def sumpair: (Any, Any) => Any = {
    case (Pair(S(x), y), Pair(u, v)) => sumpair(Pair(x, y), Pair(S(u), v))
    case (Pair(x, S(y)), Pair(u, v)) => sumpair(Pair(x, y), Pair(u, S(v)))
    case (Pair(Z, Z), Pair(u, v)) => Pair(u, v)
  }
}
```

Корневая конфигурация вычислительного графа имеет выходной формат `Pair(S(S(v130)), S(S(v131)))`. В силу введённых ограничений легко понять, что это верная гипотеза. Остаточная программа приведена в листинге 22.

Листинг 22: Результат суперкомпиляции программы `sumpair.scala`

```
case class S(x1 : Any)
case object Z
case class Pair(x1 : Any, x2 : Any)
object Main {
  def sumpair2 : (Any, Any, Any, Any) => (Any, Any) = {
    case (S(v144), v133, v134, v135) => sumpair2(v144, v133, S(v134), v135)
    case (v132, S(v145), v134, v135) => sumpair2(v132, v145, v134, S(v135))
    case (Z, Z, v134, v135) => (v134, v135)
  }
  def sumpair1 : (Any, Any, Any, Any) => (Any, Any) = {
    case (S(v137), v133, v134, v135) => sumpair1(v137, v133, S(v134), v135)
    case (v132, v133, v134, v135) => sumpair2(v132, v133, v134, v135)
  }
  def main : (Any, Any) => Any = {
    case (Pair(S(v132), S(v133)), Pair(S(v134), S(v135))) => {
      val (v126, v127) = sumpair1(v132, v133, v134, v135)
      Pair(S(S(v126)), S(S(v127)))
    }
  }
}
```

9 Руководство пользователя

Минимальным требованием к системе является наличие на компьютере приложения, поддерживающего открытие pdf-файлов (например, браузер google chrome), установленного интерпретатора языка python3, а также установленной утилиты graphviz.

Перед запуском суперкомпилятора с помощью `pip install` необходимо установить следующие библиотеки:

1. PyPDF2
2. graphviz
3. pyparsing

Запуск суперкомпилятора осуществляется командой:

```
python supercompiler.py program/path [--debug]
```

Опция `--debug` позволяет получить все этапы построения графа конфигураций. Примеры на суперкомпиляцию можно найти в папке `tests/`.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы были рассмотрены существующие подходы к специализации программ, был проведён анализ результатов работ, связанных с нахождением выходных форматов конфигураций. Основная часть работы заключалась в разработке и реализации алгоритма построения выходных форматов онлайн, в процессе построения вычислительно графа. Был разработан генератор остаточной программы, использующий информацию о выходных форматах конфигураций. Корректность алгоритмов проверялось с помощью автоматического тестирования. Тестирование показало, что доведение идеи построения выходных форматов онлайн до алгоритма прошло успешно, и все поставленные задачи были выполнены. В результате получился суперкомпилятор, имеющий в своём арсенале дополнительный способ анализа, который позволяет эффективнее решать задачи специализации и анализа свойств программ.

Благодаря значительному росту вычислительных мощностей за последнее десятилетие, концепция суперкомпиляции обладает значительным потенциалом для оптимизации программного кода и может найти применение в различных областях программирования. Это открывает пути для дальнейшего изучения и развития методов суперкомпиляции, а также их практического применения в индустрии разработки программного обеспечения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Немытых А. П. О суперкомпиляции. — 2020. — URL: http://conf.nsc.ru/files/conferences/Lyap100/fulltext/69293/69928/nemytykh_supercompilation_Lyapunov100.pdf.
2. Романенко С. А. Суперкомпиляция: основные принципы и базовые понятия // Препринты ИПМ им. М.В.Келдыша. — 2018. — Вып. 111. — С. 36.
3. Wadler P. Deforestation: transforming programs to eliminate trees // Theoretical Computer Science. — 1990. — Т. 73, № 2. — С. 231—248. — ISSN 0304-3975. — DOI: [https://doi.org/10.1016/0304-3975\(90\)90147-A](https://doi.org/10.1016/0304-3975(90)90147-A). — URL: <https://www.sciencedirect.com/science/article/pii/030439759090147A>.
4. Burstall R. M., Darlington J. A Transformation System for Developing Recursive Programs // J. ACM. — New York, NY, USA, 1977. — ЯНВ. — Т. 24, № 1. — С. 44—67. — ISSN 0004-5411. — DOI: 10.1145/321992.321996. — URL: <https://doi.org/10.1145/321992.321996>.
5. Немытых А. П. Суперкомпилятор SCP4: Общая структура. — Москва : Издательство ЛКИ, 2007. — С. 152.
6. Turchin V. F. Refal-5, Programming Guide and Reference Manual. — Holyoke, Massachusetts : New England Publishing Co, 1989. — URL: <http://www.botik.ru/pub/local/scp/refal5/>.
7. Jones N. D., Gomard C. K., Sestoft P. Partial evaluation and automatic program generation. — Prentice Hall, 1993. — (Prentice Hall international series in computer science). — ISBN 978-0-13-020249-9.
8. Futamura Y., Nogi K. Generalized partial computation. — 1988.
9. Futamura Y., Konishi Z., Glück R. Program Transformation System Based on Generalized Partial Computation // New Generation Computing. — 2002. — Т. 90. — С. 75—99.
10. Chang C.-L., Lee R. C.-T. Symbolic Logic and Mechanical Theorem Proving. — Academic Press, 1973.

11. Futamura Y., Nogi K. Generalized partial computation // Proceedings of the IFIP TC2 Workshop. — Amsterdam : North-Holland Publishing Co, 1988. — С. 133—151.
12. Higman G. Ordering by divisibility in abstract algebras // Proceedings of the London Mathematical Society. — 1952. — Т. 3, вып. 2. — С. 326—336.
13. Kruskal J. B. Well-quasi-ordering, the tree theorem, and Vazsonyi's conjecture // Transactions of the American Mathematical Society. — 1960. — Вып. 95. — С. 210—225.
14. Romanenko S. A. A Small Positive Supercompiler in Idris. — 2018. — URL: <https://github.com/sergei-romanenko/spsc-idris>.
15. Романенко С. А. Суперкомпиляция: гомеоморфное вложение, вызов по имени, частичные вычисления // Препринты ИПМ им. М.В.Келдыша. — 2018. — Вып. 209. — С. 32.

ПРИЛОЖЕНИЕ А

Таблица 3 — Методы класса ProcessTreeBuilder

Метод	Описание
<code>drivingFcall(exp, checkContractionNeed)</code>	Прогонка функции <code>f(...)</code> , если <code>checkContractionNeed == False</code> , иначе — проверка принадлежности <code>f(...)</code> классу глобальных/локальных конфигураций.
<code>loopBack(beta, alpha)</code>	Если <code>beta</code> — частный случай <code>alpha</code> (т.е. функция <code>instOf(beta, alpha)</code> из модуля matcher.py возвращает <code>True</code>) обобщает <code>beta</code> , вводя <code>let</code> -выражение, чтобы потом его зациклить на <code>alpha</code> .
<code>generalizeAlphaOrSplit(beta)</code>	Вызывается из основного цикла для осуществления обобщения в случае, если «свисток» сработал.
<code>findEmbeddedAncestor(beta, contractionNeed, drivingFuncName)</code>	«Свисток». Для его срабатывания, необходимо чтобы для конфигураций выполнялись три условия 1) гомеоморфное вложение 2) принадлежность одному классу управления 3) одинаковое имя прогоняемой функции.
<code>buildProcessTree(k)</code>	Точка входа в процесс построения графа конфигураций. Реализует основной цикл суперкомпиляции. Аргумент <code>k</code> — ограничение на размер дерева.

Листинг 23: Функция построения дерева конфигураций (6/7)

```

1  contrNeed = beta.exp.isFGHCall() and self.drivingEngine.drivingFCall(beta.exp,
   ↪  checkContractionNeed=True)
2  if beta.exp.isFGHCall():
3      alpha = self.findEmbeddedAncestor(beta, contrNeed, beta.drivingFuncName)
4      if alpha:
5          gen = self.msgBuilder.build(alpha.exp, beta.exp)
6          self.tree.render(f"dangerous embedding found. msg = {str(gen.exp)}", [alpha, beta])
7          if gen.exp.isVar():
8              self.split(beta)
9          else:
10             self.abstract(alpha, gen.exp, gen.substA)
11     return

```

Листинг 24: Функция построения дерева конфигураций (7/7)

```
1 if beta.exp.isFGHCall():
2     children = self.drivingEngine.drivingFCall(beta.exp)
3     self.tree.addChildren(beta, children)
4     for child in beta.children:
5         self.buildRecursive(child)
6     return
```

Листинг 25: Функции-обработчики гомеоморфного вложения

```
1 def abstract(self, alpha, exp, subst):
2     bindings = list(subst.items())
3     bindings.sort()
4     letExp = Let(exp, bindings, Let.Type.HE_ABSTRACT)
5     self.tree.replaceSubtree(alpha, letExp)
6     self.tree.render("dangerous embedding managed", [alpha])
7     self.manageLet(alpha)
8
9 def split(self, beta):
10    exp = beta.exp
11    args = exp.args
12    names1 = self.nameGen.freshNameList(len(args))
13    args1 = [Var(x) for x in names1]
14    letExp = Let(beta.exp.cloneFunctor(args1), list(zip(names1, args)). Let.Type.HE_SPLIT)
15    self.tree.replaceSubtree(beta, letExp)
16    self.tree.render("dangerous embedding managed", [beta])
17    self.manageLet(beta)
```

Листинг 26: Программа fac.scala

```
1 case object A
2 case object B
3 case object C
4 case object Nil_
5 case class Cons(head: Any, tail: Any)
6
7 object Main {
8     def fab : Any => Any = {
9         case Cons(A, xs) => Cons(B, fab(xs))
10        case Cons(x, xs) => Cons(x, fab(xs))
11        case Nil_ => Nil_
12    }
13
14    def fbc : Any => Any = {
15        case Cons(B, xs) => Cons(C, fbc(xs))
16        case Cons(x, xs) => Cons(x, fbc(xs))
17        case Nil_ => Nil_
18    }
19
20    def main : Any => Any = {
21        case Cons(A, xs) => fbc(fab(Cons(A,xs)))
22    }
23 }
```

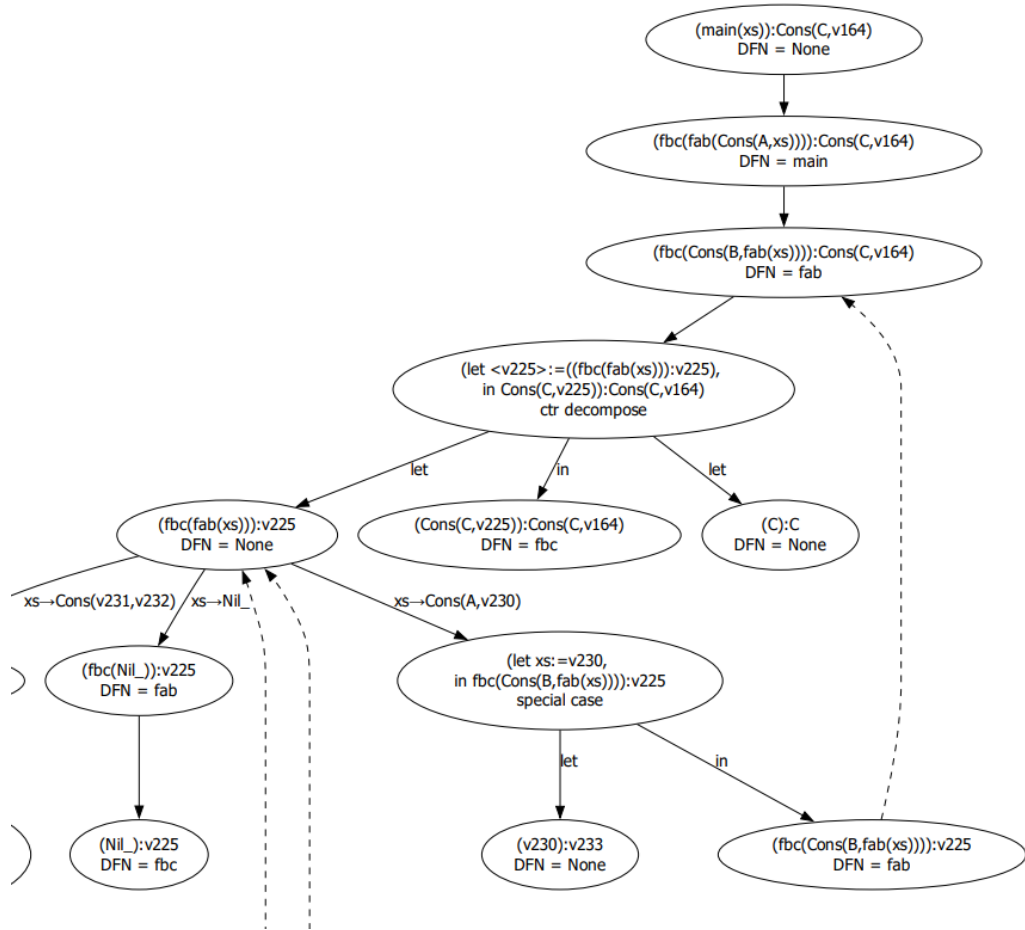



Рисунок 6 — Итоговый граф конфигурации для программы `fac.scala` (1)

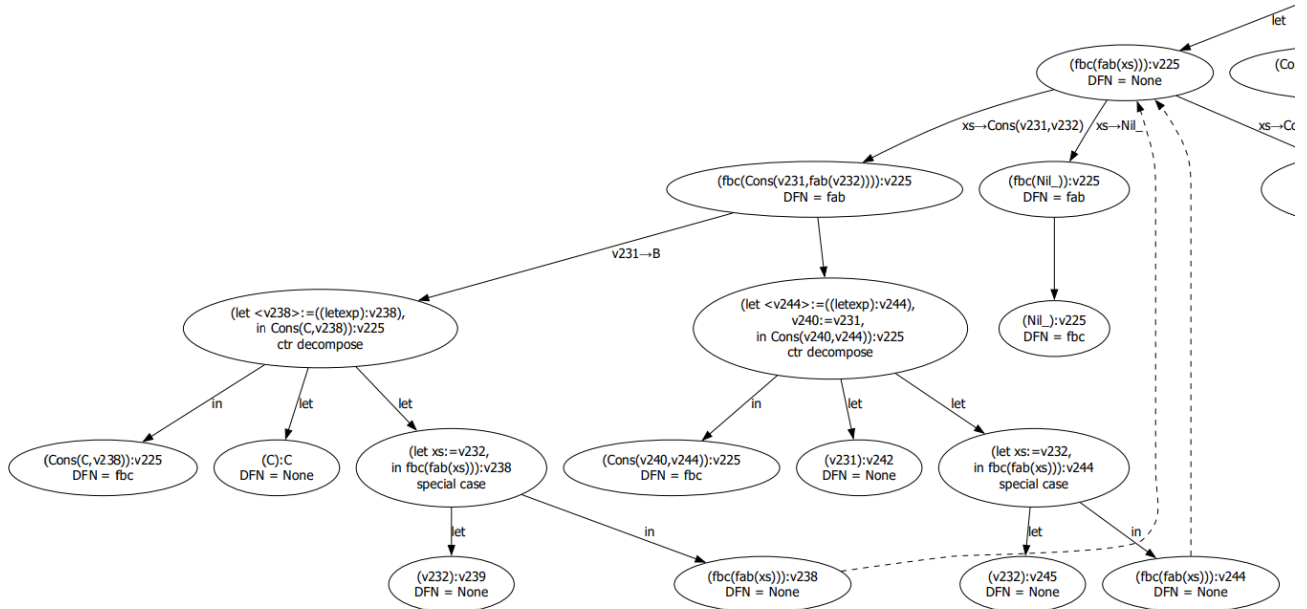


Рисунок 7 — Итоговый граф конфигурации для программы `fac.scala` (2)

Листинг 27: Результат суперкомпиляции программы `fac.scala`

```
1 case class Cons(x1 : Any, x2 : Any)
2 case object A
3 case object C
4 case object B
5 case object Nil_
6 object Main {
7   def fbc2 : Any => Any = {
8     case Cons(A, v212) => Cons(C, fbc1(v212))
9     case Cons(B, v214) => Cons(C, fbc2(v214))
10    case Cons(v213, v214) => Cons(v213, fbc2(v214))
11    case Nil_ => Nil_
12  }
13  def fbc1 : Any => Any = {
14    case v156 => fbc2(v156)
15  }
16  def main : Any => Any = {
17    case Cons(A, v156) => Cons(C, fbc1(v156))
18  }
19 }
```